

Face Recognition-Based User Verification and Virtual Access Control System

Software-only prototype using Python, OpenCV, face recognition and a browser webcam in Google Colab

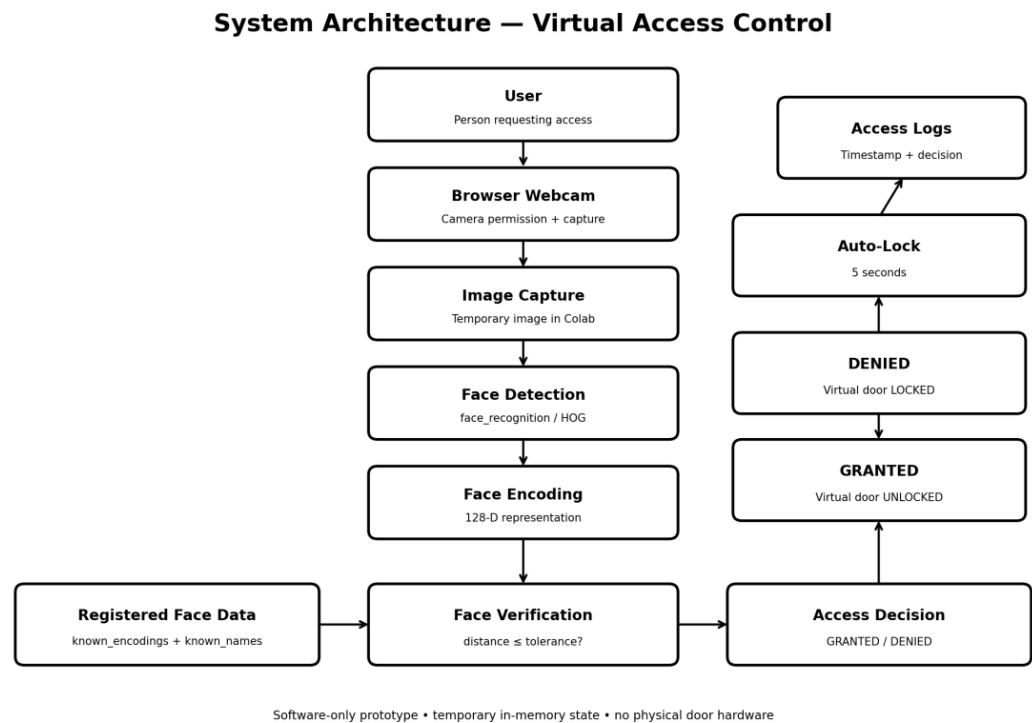


Figure 1. Overall system architecture

Prepared by: Pravallika Satya Bokka
B.Tech – Computer Science & Engineering (Data Science)
Raghu Engineering College, Visakhapatnam

Project Type	Software Prototype / Academic Project
Execution Environment	Google Colab + Browser Webcam
Primary Language	Python
Access Control	Virtual door simulation
Data Storage	Temporary / In-memory

1. Abstract

This project presents a software-only prototype for face-based user verification and virtual access control. A browser webcam captures a single image, the system detects a face, generates a face encoding, and compares it with temporarily registered face encodings. A sufficiently close match results in access being granted and the virtual door being marked as unlocked; otherwise access is denied and the virtual door remains locked. The system also records access attempts in an in-memory log. The implementation is designed for demonstration and learning rather than production security or physical door control.

2. Problem Statement

Traditional access-control demonstrations often require dedicated hardware such as cameras, controllers, relays and physical locks. For an academic prototype, a software simulation can demonstrate the same decision flow without requiring hardware. The project provides a clear and reproducible demonstration of identity verification, access decisions and logging using a browser webcam and Google Colab.

3. Aim and Objectives

Aim: Develop a face-recognition-based user verification prototype with a virtual access-control workflow.

- Capture webcam images through the browser.
- Register users using face encodings.
- Verify captured faces against registered encodings.
- Grant or deny access based on the configured matching tolerance.
- Simulate virtual door unlock and automatic relock.
- Maintain temporary access history.

4. Scope and Purpose

The scope is limited to a software demonstration. It includes browser webcam capture, face detection, face encoding, matching, access decisions, virtual lock/unlock behavior and temporary access logs. It does not include a physical door, relay, Raspberry Pi, persistent biometric database, cloud storage or production-grade security controls.

5. Technologies and Tools

Technology / Tool	Role
Python	Core implementation language
OpenCV	Image processing, resizing, color conversion and visual annotations
face_recognition	Face detection, face encoding and face-distance comparison
NumPy	Numerical operations and closest-match selection
Pandas	Display of temporary access history
Google Colab	Notebook execution environment
Browser Webcam API	Captures a webcam image through the browser
GitHub	Project documentation and source-code presentation

6. System Architecture

The system follows the pipeline: User → Browser Webcam → Image Capture → Face Detection → Face Encoding → Face Verification → Access Decision → Virtual Door → Access Logs.

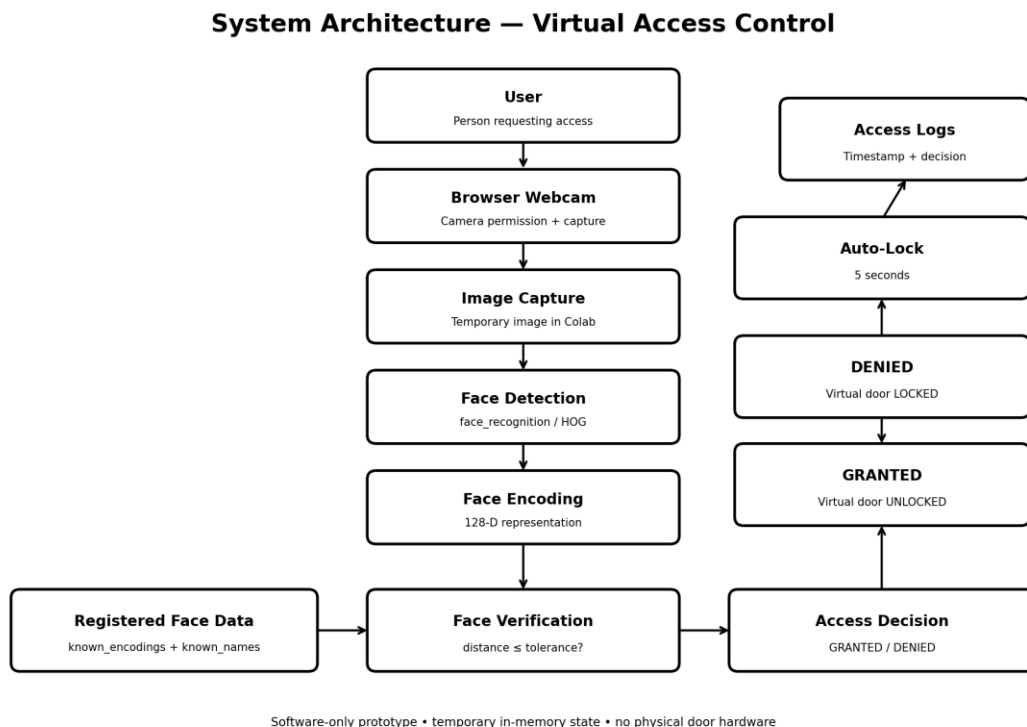


Figure 2. System architecture of the implemented software prototype.

Execution flow: Install & Import → Initialize → Capture → Register → Verify → Decide → Simulate Unlock/Lock → Log Access History.

7. Working Methodology

7.1 Initialization

The notebook creates temporary in-memory lists for registered encodings, names and access logs. The virtual door starts in the LOCKED state.

7.2 Webcam Capture

JavaScript requests browser camera access, displays a preview, captures one frame when the user presses CAPTURE PHOTO, and saves it temporarily in the Colab runtime.

7.3 Registration

The captured image is checked for exactly one face. A face encoding is generated and appended to the in-memory registered-user lists.

7.4 Verification

A new webcam image is captured and processed. Face encodings are compared with the registered encodings using face distance.

7.5 Access Decision

If the closest distance is less than or equal to `MATCH_TOLERANCE = 0.50`, the demonstration treats the identity as matched; otherwise access is denied.

7.6 Virtual Door

A granted decision sets the virtual door to UNLOCKED and starts a five-second timer. After the interval, the door is automatically locked.

7.7 Logging

Each access attempt is stored temporarily with timestamp, identity, decision, reason, face distance when available, and door status.

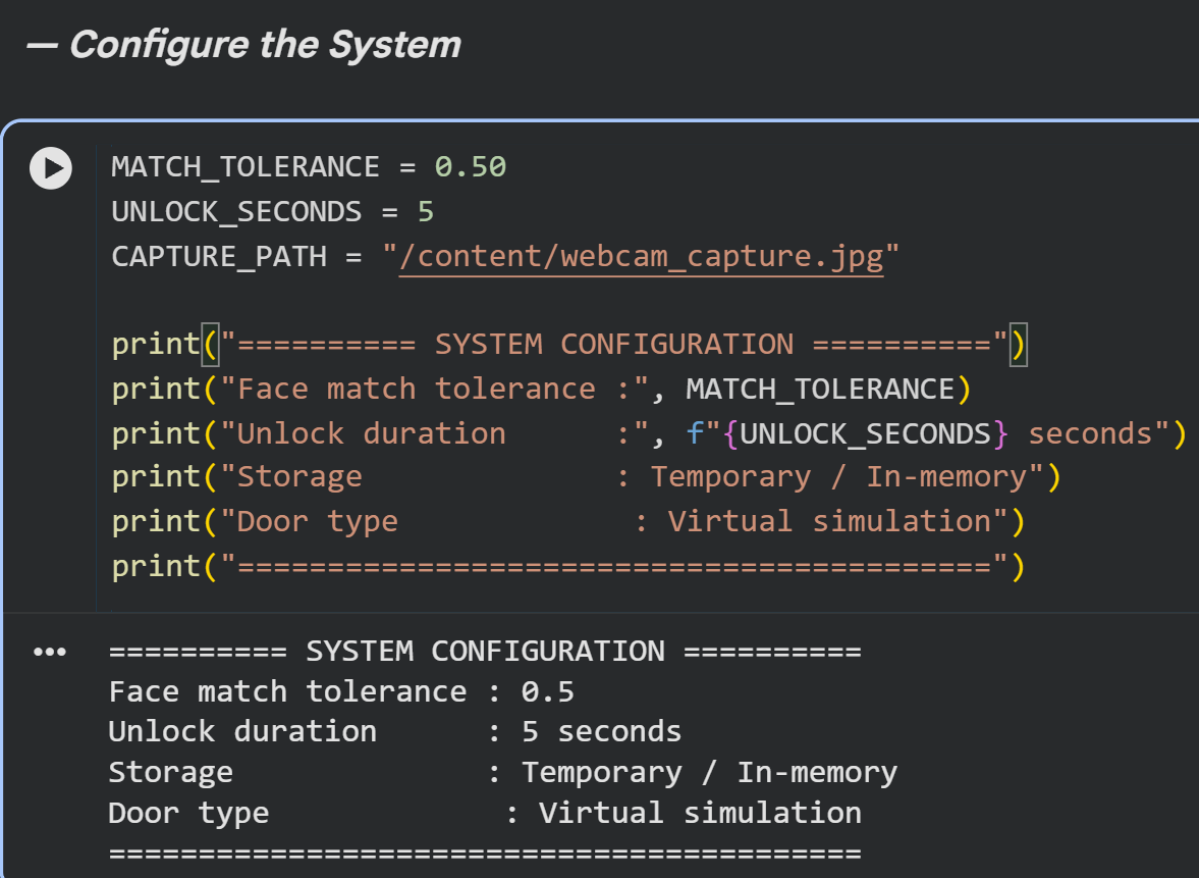
8. Implementation Details

The notebook is organized into sequential cells covering installation, imports, configuration, initialization, webcam capture, webcam testing, registration, virtual door and logging, face recognition, access verification, testing, access history and the final interactive menu.

9. Testing and Results

The following figures are the actual execution screenshots supplied for this report. They provide visual evidence of the implemented workflow from configuration and webcam capture through registration, verification, access decisions and logging.

9.1 System Configuration / Initialization



```
— Configure the System

▶ MATCH_TOLERANCE = 0.50
  UNLOCK_SECONDS = 5
  CAPTURE_PATH = "/content/webcam_capture.jpg"

print("==== SYSTEM CONFIGURATION ====")
print("Face match tolerance :", MATCH_TOLERANCE)
print("Unlock duration      :", f"{UNLOCK_SECONDS} seconds")
print("Storage                : Temporary / In-memory")
print("Door type               : Virtual simulation")
print("====")

...  ==== SYSTEM CONFIGURATION ====
     Face match tolerance : 0.5
     Unlock duration      : 5 seconds
     Storage              : Temporary / In-memory
     Door type            : Virtual simulation
     =====
```

Figure 3. System configuration showing `MATCH_TOLERANCE = 0.50`, five-second unlock duration, temporary in-memory storage and virtual door simulation.

9.2 Webcam Capture



Figure 4. Successful browser-webcam capture used as the image input for the system.

9.3 User Registration

```

===== SYSTEM MENU =====
1. Register a new person using webcam
2. Webcam face access check
3. View access history
4. Lock virtual door
5. Exit
Choose 1-5: 2
Starting webcam access check.
Capture one face when the preview appears.
Webcam photo captured. Recognizing...
VIRTUAL DOOR: UNLOCKED (auto-lock in 5 seconds)
\n===== ACCESS RESULT =====
Person       : Hemanth
Decision     : GRANTED
Reason       : Enrolled face matched
Face distance: 0.1954
Door status  : UNLOCKED

```

Figure 5. User registration stage using a captured face. The enrolled identity is stored temporarily in memory.

9.4 Face Verification

```

\n===== ACCESS RESULT =====
Person       : Hemanth
Decision     : GRANTED
Reason       : Enrolled face matched
Face distance: 0.1954
Door status  : UNLOCKED
=====

```

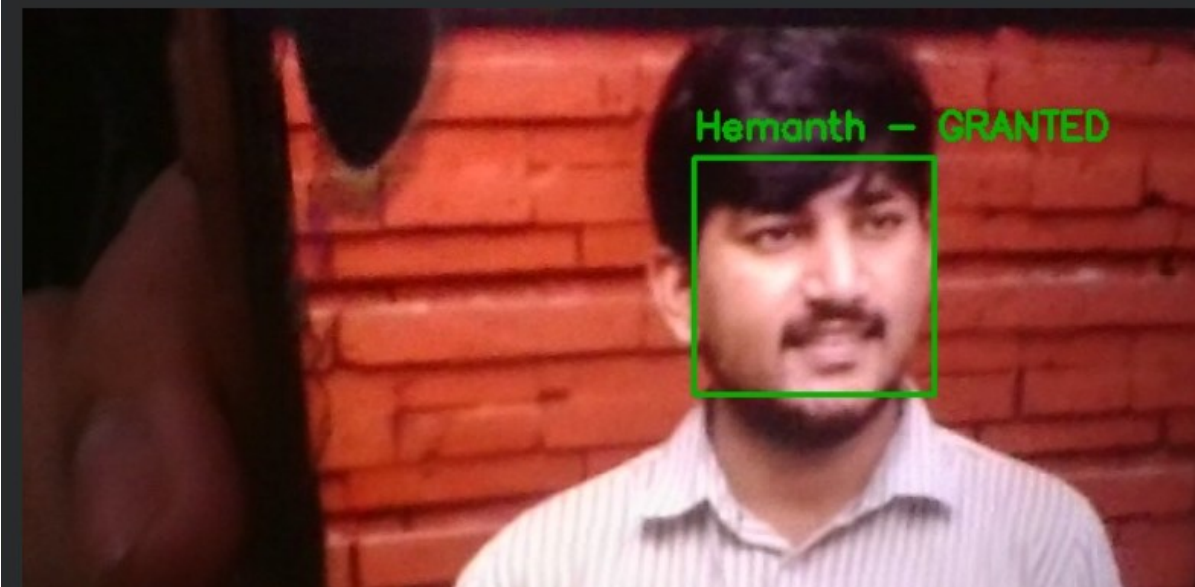


Figure 6. Face verification output showing the detected face and the enrolled identity with a GRANTED result.

9.5 Access Granted

Choose 1-5: 3

	Timestamp	Identity	Decision	Reason	Face Distance	Door Status	
0	2026-10-03 15:51:28	Hemanth	GRANTED	Enrolled face matched	0.1954	UNLOCKED	

Figure 7. Successful verification: Hemanth is recognized, the decision is GRANTED, face distance is 0.1954, and the virtual door is UNLOCKED.

9.6 Access Denied

Person	: Unknown
Decision	: DENIED
Reason	: No sufficiently close match
Door status	: LOCKED

Figure 8. Negative verification case: an Unknown identity is DENIED because no sufficiently close match is found, and the virtual door is LOCKED.

9.7 Access History / Logs

Timestamp	Identity	Decision	Reason	Face Distance	Door Status
...	Pravallika	GRANTED	Enrolled face matched	...	UNLOCKED
...	Unknown	DENIED	No sufficiently close match	...	LOCKED

Figure 9. Access-history output showing recorded GRANTED and DENIED events with identity, decision, reason and door status.

9.8 Test Case Summary

Test	Action	Observed / Expected evidence
TC-01	System configuration	Configuration values and virtual simulation state displayed.
TC-02	Webcam capture	Webcam capture completed successfully.
TC-03	User registration	A face is captured and registered as an enrolled user.
TC-04	Face verification	Registered identity is matched and a face box is displayed.
TC-05	Access granted	Decision is GRANTED and virtual door becomes UNLOCKED.
TC-06	Access denied	Unknown identity is DENIED and virtual door remains LOCKED.

10. Limitations

- Software-only prototype; it does not control a physical door, relay or Raspberry Pi hardware.
- Face data and access logs are stored temporarily in memory and may be lost when the Colab runtime resets.
- The demonstration uses a single captured webcam image for each access attempt rather than continuous live-video recognition.
- The matching tolerance is a demonstration parameter and should not be treated as a production security threshold.
- The system does not implement liveness detection, anti-spoofing, multi-factor authentication or secure biometric storage.
- Browser and operating-system camera permissions are required for webcam operation.

11. Future Enhancements

- Add persistent and secure user storage with appropriate biometric-data protection.
- Add liveness detection and anti-spoofing mechanisms.
- Add role-based access control and multi-factor authentication.
- Integrate with physical access-control hardware in a controlled laboratory environment.
- Add continuous video processing and performance monitoring if real-time operation is required.
- Add stronger audit logging, secure deployment and privacy controls.

12. Conclusion

The project demonstrates an end-to-end face verification workflow in Google Colab, combining webcam capture, face recognition, virtual access decisions, automatic relocking and temporary logging. The supplied execution evidence demonstrates the major workflow stages, including configuration, webcam capture, registration, successful verification, denied verification and access history. The project remains explicitly a software-only academic prototype and does not claim physical door control or production-grade biometric security.